

PS1 - Offline Privacy-Preserving Hindi Voice Assistant on Raspberry Pi (ARM Edge Deployment)

Bharat AI-SoC Student Challenge 2026

Team Members

- Pushkar Chaturvedi
- Rishabh Jain

Mentor

- Mr. Ajay Kumar

Jaypee University of Engineering and Technology

Problem Statement :

Most modern voice assistants depend on **cloud processing**.

This leads to several issues:

- **Privacy concerns** (user audio sent to cloud)
- **Internet dependency**
- **Higher latency**

Additionally, running speech AI on **low-power edge devices** remains challenging.

Goal :

Build a **low-latency, privacy-preserving Hindi voice assistant** running entirely on a **Raspberry Pi CPU** without cloud services (with sub 2 sec Latency).

Our Solution :

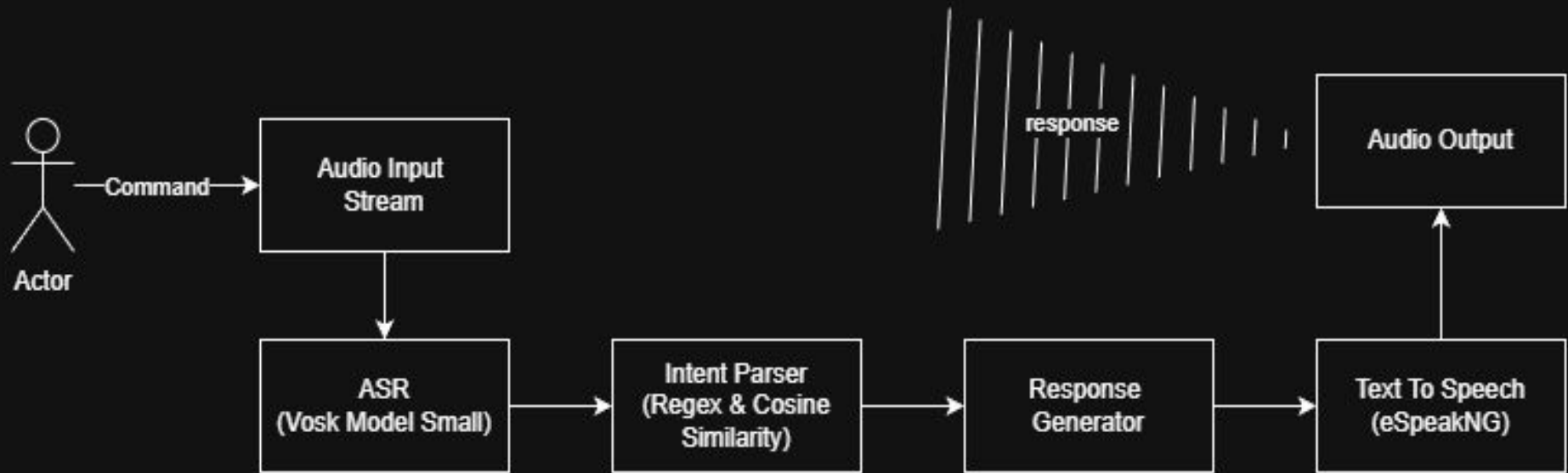
We developed a **fully offline Hindi Voice Assistant** optimized for **ARM-based edge devices**. The assistant processes speech entirely **on-device** using a modular pipeline.

Key capabilities:

- Offline Hindi Speech Recognition
- Local Intent Recognition
- Offline Text-to-Speech
- Low-latency streaming pipeline
- Complete data privacy

All processing runs locally on the Raspberry Pi CPU. No Internet Required.

System Architecture :



General Pipeline Structure

Hardware & Software Stack :

Hardware

- Raspberry Pi 4 Model B (ARM CPU)
- I2S Microphone (INMP441 Module)
- Speaker output

Software Stack

- Python 3.10+
- PyAudio for audio input
- **Vosk ASR (Hindi model)**
- Custom NLP engine
- **eSpeak-NG TTS**
- Scikit-learn (TF-IDF similarity)

Designed specifically for **CPU-only inference without accelerators**.

Custom NLP engine :

Hybrid Intent Recognition

Two-step approach designed for **edge efficiency**.

Step 1 – Rule-Based Parsing

- Regex pattern extraction
- Keyword mapping
- Text normalization
- Filler word removal

Step 2 – Cosine Similarity Fallback

- TF-IDF vectorization + semantic similarity

Helps handle variations like:

टाइमर लगाओ

10 सेकेंड का टाइमर लगा

टाइमर शुरू करो

This improves **robustness without heavy ML models**.

Performance & Benchmark :

Edge Performance Metrics

Streaming benchmark results:

- CPU usage: **~53%**
- Memory usage: **~162 MB**
- ASR processing latency: **~250 ms**

streaming_latency_20260214_120523.log X

streaming_latency_20260214_120523.log

```
1 === Streaming Latency Benchmark (Competition Mode) ===
2 CPU usage           : 45.4 %
3 Memory usage        : 155.1 MB
4
5 Speech duration      : 1626.2 ms
6 Processing latency   : 236.9 ms <-- Competition Metric
7 TTS playback duration : 3876.9 ms
8 Full completion time : 5740.0 ms
9
10 ASR Output : स्टोरेज
11 Intent     : DISK_STATUS
```

▶

streaming_latency_20260216_152010.log X

streaming_latency_20260216_152010.log

```
1 === Streaming Latency Benchmark (Competition Mode) ===
2 CPU usage           : 63.6 %
3 Memory usage        : 160.5 MB
4
5 Speech duration      : 2970.4 ms
6 Processing latency   : 250.2 ms <-- Competition Metric
7 TTS playback duration : 1829.7 ms
8 Full completion time : 5050.3 ms
9
10 ASR Output : आज दिन क्या है
11 Intent     : DAY
```

Key Features :

System Capabilities

- Wake-word activation
- Timer commands
- Time & date queries
- Fully offline ASR + TTS
- Streaming low-latency inference
- Modular pipeline architecture

Key Challenges :

- **Running Speech AI on Edge Hardware**

Speech models are typically **resource intensive**, while Raspberry Pi has limited compute.

Constraints:

- CPU-only inference
- Limited RAM
- No GPU acceleration

Solution:

- Lightweight **Vosk Hindi ASR model (~50 MB)**
- Streaming inference pipeline
- Efficient modular architecture

Key Challenges :

- **Hindi Text Normalization**

Hindi speech transcription can produce multiple variations of the same word due to **nukta characters and orthographic differences**.

Examples :

- ज़ → ज
- फ़ → फ
- आवाज़ → आवाज

- **Number Normalization for Commands**

Speech recognition models often output **numbers as words**, which creates issues for commands such as timers.

Example :

"दस सेकंड का टाइमर लगाओ"

But the system must interpret this as:

"10 सेकंड का टाइमर लगाओ"

Key Challenges :

- **Maintaining Low Latency**

Sub-2 second response time

Optimizations:

- **Streaming ASR inference**
- **Lightweight NLP pipeline**
- **CPU-friendly architecture**

Result:

- **~250 ms ASR processing latency**
- **~8 s full interaction including speech playback**